

AI Kubernetes Troubleshooting Agent: Master Investigation Guide

From Zero to Hero: Understanding the Architecture, Logic, and Code

1. What is the Target?

The goal of this project is to build an Intelligent SRE (Site Reliability Engineer).

In a traditional DevOps world, when a Kubernetes cluster fails, a human engineer has to:

1. Open a terminal.
2. Run `kubectl get pods` to find what's broken.
3. Run `kubectl logs` to see the error.
4. Run `kubectl get events` to find "hidden" infrastructure issues.
5. Try to guess the root cause based on experience.

Our Target is to automate all 5 steps. We built a system that collects the evidence (The Investigation Layer), reasons about it (The AI Engine), and presents a fix (The Dashboard).

2. Layman's Explanation (The "Hospital" Analogy)

Imagine a giant hospital (The Kubernetes Cluster) where thousands of patients (The Pods) live.

- The Junior Nurse (Investigation Layer): Every 5 minutes, a nurse goes room to room. They check if the patient is breathing (Pod Status), read their medical charts (Logs), and check if any alarms went off during the night (Events). The nurse doesn't know *how* to fix the patient, but they write everything down in a notebook.
 - The Expert Doctor (AI Reasoning Engine): The nurse hands the notebook to a world-class doctor (OpenRouter/Llama 3). The doctor looks at the data and says, "Aha! The patient isn't breathing because they didn't get their medicine (Missing Environment Variable)."
 - The Reception Desk (Dashboard): The doctor's notes are posted on a screen where the family (The User) can see exactly what's wrong and how to fix it.
 - The ID Badge (Authentication): Only people with a special badge (InsForge Login) can enter the hospital and see the patient's data.
-

3. System Flowchart (The Journey of a Request)

```
[ USER ] -> Click "Investigate"
! "
[ FRONTEND (Next.js) ] -> Sends POST request to Backend
! "
[ BACKEND (FastAPI) ] -> Creates a "Running" ticket in Database
! "
[ KUBECTL EXECUTOR ] -> Runs terminal commands on the Cluster
! "
[ EVIDENCE COLLECTORS ] -> Gathers Pod Status, Logs, Events, and Networking
! "
[ PROMPT BUILDER ] -> Bundles all evidence into a "Letter" for the AI
! "
```

```
[ OPENROUTER AI ] -> Reasons about the evidence and returns a JSON fix
!""
[ INSFORGE SERVICE ] -> Saves the Final Diagnosis and sends a Realtime Shout
!""
[ FRONTEND UI ] -> Updates the screen live with the "Root Cause" card
```

4. Project Tree Structure

Here is how the "Brain" of your project is organized:

```
ai-kubernetes-agent/
% % % backend/                # The "Engine Room" (Python)
%   % % % ai/                  # AI Logic
%   %   % % % client.py        # Talks to OpenRouter
%   %   % % % prompts.py      # How we talk to the AI
%   % % % core/
%   %   % % % config.py        # Handles Secrets and Env Variables
%   % % % kubernetes/         # The "Hands" (Junior DevOps)
%   %   % % % executor.py      # Runs terminal commands
%   %   % % % pod_inspector.py # Scans for broken Pods
%   %   % % % ...              # Other inspectors (Logs, Events, etc.)
%   % % % services/           # The "Orchestrator"
%   %   % % % ai_service.py    # Links AI to Investigation
%   %   % % % insforge_service.py # Links DB/Realtime to App
%   %   % % % investigation_service.py # Runs all checks in order
%   % % % main.py              # The Front Door (API Endpoints)
% % % frontend/               # The "Control Panel" (Next.js/TypeScript)
%   % % % src/app/
%   %   % % % login/           # Secure Login Screen
%   %   % % % signup/         # Account Creation & Email Verification
%   %   % % % page.tsx        # The Main Dashboard
% % % migrations/            # The "Filing Cabinet" (SQL)
%   % % % ..._create-tables.sql # Defines how data is stored
% % % docker-compose.yml     # The "Instruction Manual" for the entire stack
```

5. Line-by-Line Code Breakdown

A. The terminal "Hands" (`backend/kubernetes/executor.py`)

This is how Python executes commands on your computer.

```
result = subprocess.run(full_cmd, capture_output=True, text=True, check=False)
```

- `subprocess.run`: This starts a separate process to run a command (like `kubectl`).
 - `capture\_output=True`: This tells Python to "listen" to what the command says and save it.
 - `text=True`: Ensures the output comes back as a string (words) instead of binary bytes.
 - `check=False`: This is critical. If `kubectl` fails (e.g. cluster is down), we don't want our whole server to crash. We want to catch the error ourselves.
-

B. The AI Brain (`backend/ai/client.py`)

```
async with httpx.AsyncClient(timeout=60.0) as client:
    response = await client.post(self.base_url, headers=headers, json=payload)
```

- `async with`: This allows the server to keep working on other things while waiting for the AI to "think."
 - `timeout=60.0`: We give the AI 60 seconds because diagnosing complex clusters takes time.
 - `headers`: This contains your API Key. It's the secret handshake that lets you talk to OpenRouter.
-

C. The API Doorway (`backend/main.py`)

```
@app.post("/investigate")
async def investigate(payload: dict = None):
    investigation_id = await insforge_service.create_investigation(user_id)
    investigation_data = await
    investigation_service.run_full_investigation(investigation_id)
```

- `@app.post`: This defines an "endpoint." When the frontend sends data here, this function triggers.
 - `create\_investigation`: We create the database row first. This is like giving a patient a hospital ID wristband before the exam starts.
 - `run\_full\_investigation`: This starts the chain reaction of all the nurses and doctors.
-

D. The Dashboard Listener (`frontend/src/app/page.tsx`)

```
insforge.realtime.on('progress', (payload) => {
    setProgress((prev) => [...prev, payload.message]);
});
```

- `insforge.realtime.on`: This is like a radio tuned to a specific frequency.
 - `payload.message`: Every time the backend says "I'm checking pods!", this function hears it and adds it to the list on your screen.
-

6. Expected Outputs (What you will see)

When you Click "Investigate Cluster":

1. Terminal Logs:

```
```text
backend-1 | Starting full investigation for ID: 550e8400...
backend-1 | Executing: kubectl get pods -A
backend-1 | AI Reasoning...
```
```

2. UI Progress Log:

- Checking Pods...
- Reading Logs...
- AI Reasoning...
- Root Cause Found.

3. The Diagnosis Card:

- Root Cause: `ImagePullBackOff`
 - Explanation: The image `nginx:latest-wrong` does not exist in the registry.
 - Fix: Change the image tag in your deployment to a valid version like `nginx:1.21`.
 - Command: `kubectl set image deployment/web nginx=nginx:1.21`
-

7. From Zero to Hero: Learning Path

If you want to become a master of this code, learn these 3 concepts:

1. FastAPI & Async: Learn why we use `await`. It's the secret to making high-performance servers that can handle thousands of users.
2. Prompt Engineering: Look at `backend/ai/prompts.py`. Notice how we tell the AI to "Act as a Senior SRE." The better your instructions, the smarter the agent becomes.

3. State Management: In `page.tsx`, look at how `useState` changes the UI. When `investigating` is `true`, the button hides and the progress log appears. This is how modern web apps feel "alive."

Congratulations! You have built a production-style AI Agent.

Save this document as a PDF to keep it as your technical manual!